

Billigskadedyr.dk — Performance Diagnosis

Prepared 8 September 2026 · All measurements taken live from Copenhagen · Read-only WooCommerce REST API + public HTTP probing · No server or admin access used

Headline: Search is not slow because of search. Every uncached request on the site costs 1.5–3.6 s, and search is simply the one page a cache can never serve. A request that boots WordPress and returns *one byte* — no query, no template, no rendering — takes **1.46 s**. That figure is the bottleneck.

1. Measurements

Request	TTFB	What it tells us
Home page (Varnish HIT)	0.09 s	Cache layer works
Home page (cache-buster, MISS)	3.63 s	True cost, normally hidden
Shop page (MISS)	1.96 s	—
?s=rotte — 66 results	2.50 s	—
?s=zzzznomatch — 0 results	2.16 s	Nearly as slow with nothing to find → the SQL is not the problem
admin-ajax.php?action=zzz_nonexistent → returns 1 byte, HTTP 400	1.46 s	Pure plugin bootstrap. Stable across 3 runs. Healthy WooCommerce: 0.15–0.30 s
?wc-ajax=get_refreshed_fragments	1.81 / 3.03 s	Never cached. Fires on the first page view of a session (see §3.2)
Static asset (jQuery)	0.17 s	Network and CDN are fine

Search time budget

1.46 s	plugin bootstrap	(measured, no query, no render)
0.70 s	search query + theme render	(derived)

2.16 s	zero-result search	<- measured
2.50 s	66-result search	<- measured

Roughly **two thirds of every search is spent before WordPress looks at a single product**. The catalogue is only 205 products — the query itself is trivial.

Concurrency: the server saturates at ~6 users

6 simultaneous searches	TTFB
rotte / kakerlak / mus	3.71 s / 4.00 s / 4.18 s
myrer / muldvarp / fluer	4.59 s / 4.69 s / 6.09 s

Six concurrent visitors push the slowest response to 6 s. Real traffic plus crawlers will be worse. This points at the PHP-FPM worker pool.

2. Environment

Item	Value
WordPress / WooCommerce	7.1 / 11.0.0 (11.1.0 available)
PHP / MariaDB / Server	8.3.33 / 10.6.27 / Apache 2.4.68 on Cloudways
WP memory limit	1 GB (WP_MEMORY_LIMIT ; PHP's own <code>memory_limit</code> is not exposed by the API) — not a constraint
Object cache	Object Cache Pro, active and working
Page cache	Breeze + Cloudways Varnish + Cloudflare
Active plugins	40
HPOS (High-Performance Order Storage)	Disabled — 13,242 orders stored in <code>wp_posts</code>
Database size	912 MB data + 227 MB index

The infrastructure is not the problem. The application layer is.

3. Root causes, ranked

1. Forty plugins → 1.46 s of boot on every dynamic request

Measured on a request that renders nothing. This is raw PHP include and hook registration, not database work — Object Cache Pro is active and healthy, so this is not cache misses.

2. The cart-fragments request costs 1.5–3 s and is never cached

`?wc-ajax=get_refreshed_fragments` measured at 1.81 s and 3.03 s server-side. WooCommerce caches the result in the browser's `sessionStorage` and only refetches when the cart hash changes or storage is empty — so it fires on the **first page view of each session**, not on every navigation. On a site dominated by new visitors that still means a 2–3 s blocking request on close to every landing.

The response carries exactly one fragment, `div.widget_shopping_cart_content` — the mini-cart dropdown. Two to three seconds to refresh a dropdown that few visitors open. **Fix in section 5.**

3. The server saturates at low concurrency

See the table above. PHP-FPM worker count and plan size need review in the Cloudways panel.

4. The page cache does not cache any URL with a query string

Verified by request: three identical requests to `/shop/` all return `x-cache: HIT`, while three identical requests to `?s=rotte` — and to any `?param=` URL — all return `x-cache: MISS`. Clean paths cache; query strings never do. **This, not cookies, is why search is always slow.**

Every uncached response does set `woocommerce_cart_hash` and `wmc_current_currency`, but this happens equally on the home, shop and product pages, which cache normally — so the cookies are not blocking the cache. The cart hash is a constant empty-cart value, and no `wp_woocommerce_session_*` cookie is issued on a plain page view, so ordinary searches do *not* create session rows. **Fix in section 6.**

5. HPOS is off, with 13,242 orders in wp_posts

`HPOS_enabled: False`, `order_datastore: WC_Order_Data_Store_CPT`. 13,242 orders and 150 refunds share a table with your 205 products, and `wp_postmeta` is 80 MB data + 53 MB index.

Set expectations correctly: this is not a frontend speed fix. The zero-result search timing already proved SQL is not the bottleneck. HPOS mainly speeds up the admin order list and stops these tables growing with every sale. It is also the riskiest change on this list. Two lower-risk actions capture much of the same value — see section 7.

One thing to resolve before planning a migration: the database already contains `wp_wc_orders` and the full set of HPOS tables under a `wp_` prefix. Someone has created these before. Find out whether a previous migration was attempted and abandoned.

6. Roughly 620 MB of dead database, in two distinct patterns

The database holds **189 tables**. Two clusters account for nearly all the waste.

Pattern 1 — 42 tables owned by plugins that are not installed (617 MB). Not a handful; Wordfence alone left 25 tables behind.

Owner (not in active plugin list)	Tables	Size
WPML	1	519.6 MB
WP Mail SMTP	2	69.6 MB
Wordfence	25	14.1 MB
WP Import Export	1	4.5 MB
WC Google Product Feed (the redundant feed plugin)	2	3.8 MB
MonsterInsights	1	2.5 MB

Yoast's tables also match a dead-plugin pattern but **Yoast is active — leave those alone.**

Pattern 2 — an entire orphaned WordPress schema. The live prefix is `www0_`, but the database also carries **51 tables prefixed** `wp_`: `wp_posts`, `wp_users`, `wp_options`, and every WooCommerce table including `wp_wc_orders`. A migration or staging restore that was never cleaned up. Only ~3 MB, but 51 tables of ambiguity for anyone working on this database.

Pattern 3 — the one that touches the bootstrap. `www0_options` is 6.47 MB. Its autoloaded slice is read on *every single request*, so it is part of the 1.46 s. Measuring it is the highest-value query of the three (section 7).

Also present but legitimate: `usermeta` 88 MB + `users` 27 MB. Too large for this catalogue, but it is customer data — diagnose before touching (section 7).

4. Plugin audit

Each verdict below is based on what the plugin actually emits on the live site, or on the WooCommerce REST API — not on the plugin's name or description.

Confirmed on 8 Sep 2026: qTranslate-XT cannot simply be deactivated. Doing so produced a WordPress critical error and took the site down. It has been reverted; the site is confirmed healthy again (home, shop, product and search pages all return HTTP 200 with no error page). Its content is stored with inline language markers, so removing it is a content migration rather than a deactivation.

Safe to disable — verified producing no output

Plugin	Evidence	Risk
WebP Express	Product PNG returns <code>image/png</code> , 12,475 bytes — <i>identical</i> with and without <code>Accept: image/webp</code>	None. It is delivering nothing.
MC4WP: Mailchimp	Zero <code>mc4wp</code> references in any rendered page	None, if no newsletter form exists. Grep pages first.
Preview E-mails for WooCommerce	Admin-only dev tool	None.
Advanced Editor Tools + Classic Editor	Admin-only editor UI	None on frontend. Keep only if editors depend on them.
User Role Editor	One-time config tool; roles persist in the DB	None.
Loco Translate	Admin-only; the <code>.mo</code> files it wrote stay on disk	None. Reactivate when editing strings.
ManageWP - Worker	Remote management agent	Low — unless it is your only backup mechanism. Confirm first.

Duplicated functions — pick one of each

Two product feed plugins: `?ctxfeed=1` returns HTTP 200 with 171 KB of live feed — CTX Feed is working. Four plausible URLs for WooCommerce Google Product Feed all returned 404. Confirm in Merchant Center which URL Google actually fetches, remove the other, then update the survivor (CTX Feed is on 6.6.45 against a current 8.0.15).

Three analytics stacks: AWCA, Independent Analytics and gtag/GTM all load on the page, with two `independent_analytics_*` tables taking per-pageview writes. GTM alone covers this.

Do not touch — verified load-bearing

Plugin	Verified evidence
qTranslate-XT	Content is stored with inline <code>[:da]</code> markers — one leaks into the rendered product page. Deactivating fatals the site.
ePay (Apple Pay, MobilePay, card) + EAN	All four gateways report <code>enabled=true</code>
Tiered Price Table	<code>tiered-pricing-dynamic-price-wrapper</code> renders 9× on a product page
Variation Swatches	Active on product pages
Shipmondo, Weight Based Shipping	Live DK and SE shipping zones configured
Emærket	Trust badge widget loads from emaerket.dk
Partner-ads	Affiliate tracking JS live — removing it breaks attribution and costs revenue
Redirection	The <code>?lang=sv</code> → 301 may run through it
Breeze, Object Cache Pro	These are what is currently saving you
Both custom <code>billigskadedyr-*</code> plugins	Frontend and stock/pricing logic
Code Snippets	Deactivating silently kills every snippet inside. Export and read them first.

Needs a business decision

CURCY Multi Currency — renders CSS on every page and sets a cookie that helps make pages uncacheable, but only DKK prices appear anywhere and store currency is DKK. If the SEK business moved to billigskadedjur.se, this is dead weight.

avecdo — nothing identifiable renders from it, but a server-side integration cannot be ruled out from outside. Ask whoever owns the contract.

5. Fix #1 — cart fragments

This is the first item to ship: smallest change, largest immediate effect, fully reversible.

The snippet

Paste the following into **Code Snippets**. Do *not* include a `<?php` opening tag — Code Snippets adds it, and pasting one causes a parse error.

```
add_action( 'wp_enqueue_scripts', function () {  
    // WooCommerce off -> is_cart() undefined -> white screen. Guard it.  
    if ( ! function_exists( 'is_cart' ) ) {  
        return;  
    }  
    if ( is_cart() || is_checkout() || is_account_page() ) {  
        return;  
    }  
    // dequeue, never deregister – XT ATC declares this as a dependency  
    wp_dequeue_script( 'wc-cart-fragments' );  
}, 20 );
```

Snippet settings

Field	Value
Scope	Front-end only
Title	Perf: dequeue wc-cart-fragments (except cart/checkout/account)
Description	Removes the 1.5–3 s <code>?wc-ajax=get_refreshed_fragments</code> request. Delete once plugin bootstrap is fixed.

Why each line is there

- `function_exists( 'is_cart' )` — `is_cart()` is a WooCommerce function. If WooCommerce is ever inactive (failed update, plugin conflict, an audit step) the snippet would fatal on every page render, turning a recoverable state into a white screen.
- `is_account_page()` — the `order-pay` and `add-payment-method` endpoints live under my-account and touch cart state.
- Dequeue, not deregister** — the XT Ajax Add To Cart bundle declares `wc-cart-fragments` as a dependency. Deregistering removes it from the dependency graph and WordPress will then silently refuse to print XT's script, breaking ajax add-to-cart with no error. Dequeue leaves the registration intact and only skips the output.
- Priority 20** — WooCommerce enqueues at the default priority 10, so this must run after it.

Known regression. A visitor who returns with items already in their cart will see an empty mini-cart dropdown until they load `/kurv/`. Adding to cart still works and still updates correctly, because XT Ajax Add To Cart returns its own fragments in the add-to-cart response (`singleRefreshFragments` is enabled in its config). Given that `woocommerce_sessions` is only 3 MB and largely bot traffic, few real visitors should be affected.

A cookie-gated variant exists that avoids this regression by loading fragments only for visitors who already have a cart. It is roughly 25 lines, because `wc_cart_fragments_params` is localized onto the `wc-cart-fragments` handle — dequeuing the handle drops the params too, so they must be re-emitted on another handle or the lazily-loaded script throws. Not recommended unless someone actually reports the empty mini-cart.

Verifying it worked

1. Apply on staging first.
2. Open a product page in a fresh browser profile with DevTools → Network.
3. Confirm no request to `?wc-ajax=get_refreshed_fragments` appears.
4. Add a product to the cart — confirm the mini-cart still updates.
5. Load `/kurv/` and `/kasse/` — confirm the cart and checkout behave normally (fragments still load there by design).

This is an interim measure, not a permanent one. The request costs 1.5–3 s because of the 1.46 s plugin bootstrap, not because the fragment work is expensive. Once bootstrap is healthy this same request drops to roughly 0.4 s — at which point delete the snippet and let fragments run normally. That is a better outcome than maintaining a cleverer workaround.

6. Fix #2 — make search cacheable

Search cannot use the origin page cache, because that cache excludes every URL with a query string (§3.4). The site is already behind Cloudflare, so the edge is the right place to fix it.

Cloudflare Cache Rule

Setting	Value
Match	<code>http.request.uri.query</code> contains "s=" and <code>http.request.method</code> eq "GET"
Action	Eligible for cache
Edge TTL	5–15 minutes, 200 responses only
Cache key	Include only the s query parameter — ignore all others
Bypass when	cookie matches <code>wordpress_logged_in_*</code> or <code>woocommerce_items_in_cart</code>
Never match	<code>?wc-ajax=</code> , <code>?add-to-cart=</code> , <code>?remove_item=</code> , any non-GET

The cookie bypass is a safety control, not a performance tweak. Cloudflare's default cache key ignores cookies entirely. If that condition is wrong, a logged-in administrator's rendered page can be cached and served to the public. Test it explicitly: load a search while logged in and confirm the response carries `cf-cache-status: BYPASS`.

The cache-key restriction matters more than it looks. Without it, `?s=rotte&utm_source=newsletter` is a separate cache entry from `?s=rotte`, and campaign traffic shreds the hit rate.

Expected result: repeat searches for common terms (*rotte*, *mus*, *myrer*) serve from the edge in roughly 0.1 s instead of 2.5 s. First search of each term still pays full price.

The better long-term answer: a static search index

The catalogue is 205 products. A JSON index of titles, SKUs and URLs is roughly 40–60 KB — small enough to ship to the browser and search client-side. Regenerate it on product save.

- Search becomes instant, with no server request at all.
- The caching question disappears entirely.
- It allows a synonym map, so a visitor searching *bille* finds the right product even when the catalogue uses the trade term. Server-side WordPress search cannot do this.

Roughly a day of work rather than ten minutes. Do the Cache Rule this week; treat the index as the version you keep, once the plugin cleanup is done.

7. Fix #3 — database cleanup

Step 1: measure the autoload (do this first)

This is the only part of the database work that touches the 1.46 s bootstrap directly.

```
SELECT ROUND(SUM(LENGTH(option_value))/1024/1024, 2) AS autoload_mb,
        COUNT(*) AS n
FROM www0_options WHERE autoload = 'yes';

SELECT option_name, ROUND(LENGTH(option_value)/1024, 1) AS kb
FROM www0_options WHERE autoload = 'yes'
ORDER BY LENGTH(option_value) DESC LIMIT 25;
```

Under ~800 KB is healthy. If it comes back at 3–5 MB, that is a measurable slice of every request, and the second query names the plugin responsible. Options can then be flipped to `autoload = 'no'` individually.

Step 2: drop the dead tables — generate, review, then run

Do not type table names by hand. Confirm the live prefix in `wp-config.php` is `www0_`, then:

```
SELECT CONCAT('DROP TABLE `', table_name, `';') AS stmt,
        ROUND((data_length + index_length)/1024/1024, 1) AS mb
FROM information_schema.tables
WHERE table_schema = DATABASE()
      AND (table_name LIKE 'wp\_%'           -- orphaned prefix, 51 tables
           OR table_name LIKE 'www0\_wf%'    -- Wordfence, 25 tables
           OR table_name LIKE 'www0\_wpml%'
           OR table_name LIKE 'www0\_wpmailsmtp%'
           OR table_name LIKE 'www0\_wpie%'
           OR table_name LIKE 'www0\_monsterinsights%'
           OR table_name LIKE '%\_gpf\_%')
ORDER BY mb DESC;
```

Read the output. Delete any line you are not certain about. Then run what remains.

Before dropping: take a full Cloudways database backup, and additionally `mysqldump` these specific tables to a file kept off-server. `wp_wpml_mails` is a 519 MB mail archive — confirm with the business that nobody treats it as an audit trail. InnoDB with `file_per_table` reclaims disk on `DROP`, so no `OPTIMIZE` pass is needed afterwards.

Step 3: orphaned postmeta — safe, no migration

This is the low-risk alternative to HPOS that captures part of the same benefit.

```
-- count first
SELECT COUNT(*) FROM www0_postmeta pm
LEFT JOIN www0_posts p ON p.ID = pm.post_id WHERE p.ID IS NULL;

-- if meaningful, delete
DELETE pm FROM www0_postmeta pm
LEFT JOIN www0_posts p ON p.ID = pm.post_id WHERE p.ID IS NULL;
```

These rows have no parent post by definition, so nothing can reference them. On a site with 13k orders and years of plugin churn this routinely clears tens of MB from the 80 MB `postmeta` table.

Step 4: diagnose users/usermeta before deleting anything

```
SELECT COUNT(*) FROM www0_users;  
  
SELECT meta_key, COUNT(*) AS n_rows,  
       ROUND(SUM(LENGTH(meta_value))/1024/1024, 1) AS mb  
FROM www0_usermeta  
GROUP BY meta_key ORDER BY mb DESC LIMIT 20;
```

116 MB across these two tables is far too much for a 205-product catalogue. Common causes are `_woocommerce_persistent_cart_*`, session tokens, and analytics plugins storing per-user blobs. **Delete nothing until you have read that output** — some of it is real customer data.

Step 5: a business question, not a technical one

How many of the 13,242 orders are older than the five-year retention Danish bookkeeping rules require? If a large share are, exporting and then removing them clears far more rows than an HPOS migration would, with none of the plugin-compatibility risk. That decision belongs to whoever owns the accounting — but it has the best effort-to-benefit ratio of anything in this section.

8. Recommended order of work

#	Action	Expected effect	Risk
1	Disable cart fragments outside cart/checkout (§5)	Removes a 2–3 s request from every page view	Very low
2	Cloudflare Cache Rule for search URLs (§6)	Repeat searches ~0.1 s instead of 2.5 s	Low, with the cookie bypass
3	Work through the "safe to disable" list, one plugin at a time	Cuts into the 1.46 s bootstrap	Low
4	Remove the redundant feed plugin and two of three analytics plugins	Bootstrap + fewer per-view DB writes	Low, after verifying the live feed URL
5	Database cleanup: autoload audit, 93 dead tables, orphaned postmeta (§7)	Part of the bootstrap, ~620 MB reclaimed	Low — back up first
6	Review PHP-FPM workers / plan size	Fixes the concurrency cliff	Low
7	Enable HPOS (staging first) — <i>demoted</i> , see §3.5	Admin order list; not a frontend win	Medium — check ePay, Shipmondo, Sequential Order Numbers, custom plugins
8	qTranslate removal — content migration	Removes a hook on every translatable string	High — content migration
9	Only then revisit search itself	—	—

Realistic outcome of steps 1–5: bootstrap from 1.46 s to roughly 0.8–1.0 s, putting search near 1.4 s and removing 2–3 s from every page view. This is an *estimate*. The measurement protocol below gives the real number.

How to verify each step

After every single change, on staging, run:

```
curl -s -o /dev/null -w '%{time_starttransfer}\n' \
  'https://STAGING/wp-admin/admin-ajax.php?action=zzz_nonexistent'
```

That is the 1.46 s number. Watch it fall. Anything that does not move it is not worth the risk of removing. Install Query Monitor alongside and sort hooks by time — this report tells you what is *safe* to remove; Query Monitor tells you what is *expensive*. Remove what is both.

Before any change reaches production, walk the full path: add to cart → checkout with MobilePay → order confirmation → Shipmondo label. That crosses most of the load-bearing plugins at once.

9. Limits of this analysis

- **Per-plugin timings were not measurable** from outside. Only the aggregate 1.46 s is a measurement; the split across 40 plugins requires Query Monitor on staging.
- **OPcache status unverified.** The perfectly flat 1.46 s across three runs suggests it is enabled and the cost is genuinely plugin code — but confirm in the Cloudways panel.

- **PHP-FPM worker count unverified.** Inferred from the concurrency curve only.
- **Database queries not executed.** The API access used for this report is read-only and carries no database access. Every SQL statement in section 7 is written to be run by you — counts and audits first, destructive statements only after reading their output.
- **Admin settings not inspected.** Verdicts rest on live frontend output and the REST API. Plugins with purely server-side integrations (avecdo) cannot be ruled out this way.